

USER MANUAL

DyCAD

Enterprise Architecture Modeling & Simulation,
from Foundations to Full Reference

Covers application version **v0.948**

Prepared 2026-09-07

Open source — github.com/ldanberger/dycad

Contents

Foundations: Enterprise Architecture, Process Mapping & Data Modeling

- What Is Enterprise Architecture?
- Process Mapping
- Data Modeling
- Why Model All of This Together?

How DyCAD Is Built: One Dataset, Many Diagrams

- The Core Dataset: Parts and Connectors
- Views Are Windows, Not Storage
- The Catalog of Parts Common to Every Discipline
- Streams: Tagging Cross-Cutting Flows

Guided Walkthrough

- First Launch
- Loading a Populated Example
- Browsing the Parts Catalog
- Generating Views From the Dataset
- Touring the Generated Views
- Editing a Part: the Property Panel Split
- Generate Stream and Stream Templates
- Console Scripting: Building and Remapping a View
- The 3D View
- Proof: Delete a View, Regenerate It

Reference

- The Data Model
- Views & Layout
- Commands
- Filtering, Streams & Catalogs
- Property Panels
- Simulation Scripting
- ArchiMate Import
- Data Modeling (Entity-Relationship)
- Tips & Things to Know
- Keyboard Shortcuts

Appendix

- Generate View: the Full TOGAF Starter Set
- Where to Learn More

Foundations: Enterprise Architecture, Process Mapping & Data Modeling

This part assumes no prior background. If you already know what enterprise architecture, process mapping, and data modeling are, skip ahead to Part 2, where the manual gets specific to DyCAD.

1.1 What Is Enterprise Architecture?

Enterprise architecture (EA) is the discipline of describing how an organization's strategy, the work it does, the information it manages, and the technology it runs on all fit together — and using that description to plan change deliberately instead of by accident.

Think of it the way an architect's blueprints describe a building: not just what the building looks like, but its structural frame, its plumbing and electrical systems, and how those systems depend on each other. An enterprise "blueprint" does the same thing for an organization. It typically separates the description into layers:

- **Business layer** — the organization's actors, roles, business processes, business functions, and the capabilities it has (what it is able to do, e.g. "Process Customer Orders").
- **Application layer** — the software systems that support those business activities, and how those systems talk to each other.
- **Technology layer** — the infrastructure (servers, networks, platforms) the applications actually run on.
- **Motivation & Strategy layer** — the drivers, goals, and requirements that explain *why* the rest of it exists in the first place.
- **Implementation & Migration layer** — the work packages and plateaus used to get from the current state to some future state.

Two standards come up constantly in this field, and it's worth knowing the difference:

- **TOGAF** (The Open Group Architecture Framework) is a *method* — a process for developing and governing an enterprise architecture, and a standard set of view types architects are expected to produce (an organization chart, a capability map, a data flow diagram, and so on).
- **ArchiMate** is a *modeling notation* — a fixed vocabulary of element types (Business Actor, Business Process, Application Component, Data Object, ...) and the relationships allowed between them (Serving, Realization, Composition, Triggering, ...), so that a diagram drawn by one architect means the same thing to another.

DyCAD is grounded directly in the ArchiMate 3.x element/relationship vocabulary, and borrows TOGAF's standard list of view types as ready-made starting points (see [Generate View](#), Part 3). You do not need to have read either specification to use it — but recognizing the names ("Business Function," "Application Component," "Realization") will make the toolkit feel familiar rather than arbitrary.

Why organizations bother with this: without an explicit model, the relationships between "what we do," "what software we run," and "what data we manage" live only in people's heads, scattered across slide decks that go stale the week after they're presented. A connected model lets you answer questions like "*if we retire this application, which business capabilities lose support, and which data does it currently own?*" — the kind of question that is expensive to answer badly and easy to answer well once the relationships are actually recorded somewhere.

1.2 Process Mapping

Process mapping is the practice of documenting, visually, how work actually flows through an organization: what triggers it, what steps happen in what order, who or what is responsible for each step, and what it produces.

A few terms that recur constantly:

- **Business process** — a sequence of activities that together produce something of value (e.g. "Process Customer Order").
- **Business event** — something that happens and triggers a process (e.g. "Order Received").
- **Value stream** — the end-to-end path by which an organization delivers value to a customer or stakeholder, usually spanning several processes and organizational boundaries.
- **Business function** — an ongoing area of organizational activity or responsibility (e.g. "Sales Operations"), as distinct from a process, which has a beginning and an end.

Process maps are useful because they surface things a purely verbal description hides: duplicated steps, unclear handoffs between departments, a process with no owner, or a process that depends on a system nobody remembers is still running. In an enterprise-architecture context, process mapping earns its keep specifically when a process is connected back to the *capability* it realizes, the *application* that supports it, and the *data* it reads or writes — rather than existing as an island diagram nobody revisits.

1.3 Data Modeling

Data modeling is the practice of describing the structure of the information an organization cares about — independent of any one application's database schema — so that everyone agrees on what a "Customer" or an "Order" actually consists of.

The core vocabulary, usually drawn as an **entity-relationship diagram (ERD)**:

- **Entity** — a distinct "thing" the business tracks records about (Customer, Order, Product). Drawn as a labeled box/table.
- **Attribute** — a named property of an entity (a Customer has a Name, an Email); each has a data type (string, number, date, boolean, ...).
- **Primary key (PK)** — the attribute (or attributes) that uniquely identifies one row of an entity.
- **Foreign key (FK)** — an attribute on one entity that points at another entity's primary key, expressing a relationship (an Order has a CustomerId referencing Customer).
- **Cardinality** — how many of one entity relate to how many of another: One, Many, Zero-or-one, One-or-many. Conventionally drawn with **crow's-foot notation** — a forked "crow's foot" symbol at the "many" end of a relationship line, a single tick at the "one" end.

Getting this right matters because data outlives any one application. Systems get replaced; the entities they managed (a customer record, an order, an inventory item) conceptually persist and need to be re-hosted somewhere. A data model that exists independently of any one application's schema is what makes that migration tractable instead of a guessing game.

1.4 Why Model All of This Together?

The core insight

Business capabilities, processes, applications, technology, and data are not actually separate concerns — they're facets of the same underlying reality, and in most organizations they are maintained as separate, disconnected diagrams anyway: a capability map in one slide deck, a process map in another tool, a data model in a database design document, an application inventory in a spreadsheet. None of them can answer a cross-cutting question, because none of them contain the others' information.

DyCAD's premise is that all five kinds of thing — actors, capabilities, processes, applications, technology components, and data entities — belong in *one connected graph*, not five disconnected diagrams. A Data Entity can be linked directly to the Business Capability that depends on it and the Application that manages it, in the same document, using the same shared vocabulary of element types. That's what the rest of this manual is about.

How DyCAD Is Built: One Dataset, Many Diagrams

2.1 The Core Dataset: Parts and Connectors

Everything you model in DyCAD ultimately becomes one of five record types. Understanding how they relate is the single most useful thing to know before clicking around — nearly every "why did that happen" moment traces back to this.

Record	What it is	Key fields
Part	A "thing" in your architecture — an actor, capability, application, data entity, technology component, etc. Shared: the same part can appear on many views at once.	Type, Label, Model, Description, Note, Script
Connector	A relationship between two parts (Serves, Realizes, Triggers, Composition, ...). Also shared across every view that shows it.	From, To, Relationship type, Model, Note
View	A diagram — what you actually look at and edit. A view does not contain parts or connectors directly; it contains <i>placements</i> of them.	Name, View Type, layout/routing settings
ViewMember	One part or connector's appearance on <i>one specific view</i> — its position, fill color, whether it's flagged "external." The same underlying part can sit in a different position with a different color on every view that shows it, because each view keeps its own ViewMember for that part.	x/y position, Fill Color, Order, External flag
Model	A namespace parts and connectors belong to — use it to separate as-is vs. to-be architectures, different business units, or simulate independently scoped scenarios.	Just a name

The load-bearing design decision is the **Part/Connector vs. ViewMember split**. Parts and Connectors are the shared, model-level truth — the actual dataset. ViewMembers are per-diagram placement and styling only. Edit a Part's Label or Type from its property panel, and the change appears on *every* view that part happens to be placed on, because you just edited the shared record. Drag that same part to a new position on one view, and only that view's ViewMember changes — the part itself, and its appearance on every other view, is untouched.

2.2 Views Are Windows, Not Storage

Mindset change

Views are used once to input data, then are used as reports that can be discarded or quickly regenerated and remapped automatically to desired / standardized layouts. A view in DyCAD is not where your information lives. It is a *window* you open onto the shared dataset — a convenient place to enter data, connect things visually, and review a slice of the architecture. Once a Part or Connector has been created, the view that helped you create it becomes optional. You can close that view's tab, delete the view outright, or leave it open and forgotten — none of those actions touch the underlying Part or Connector records, because those records don't belong to the view. They belong to the document as a whole.

A useful analogy: a pivot table in a spreadsheet is a view onto the underlying rows of data. You can build a pivot table, delete it, and build a completely different one later — the source rows were never inside the pivot

table to begin with, so nothing is lost either way. DyCAD's views work the same way relative to Parts and Connectors.

This has a very practical consequence: instead of treating a diagram as the deliverable to protect, DyCAD treats the *dataset* as the deliverable, and treats diagrams as disposable, regeneratable snapshots of it. Several commands exist specifically to (re)build a view *from* whatever is already in the dataset, rather than requiring you to hand-place things every time:

- **Advanced > Generate View** — builds a full set of standard TOGAF-style starter views (Business Capability View, Organization Structure View, Data Management View, ...) directly from whatever Parts and plain relationships already exist in a chosen Model. Run it again later, after the dataset has grown, and it builds fresh views reflecting the current state — nothing about the first run needs to be preserved for the second to work.
- **Advanced > Generate Inventory View** — a single summary view of everything currently in the model.
- **Add Existing** (Commands panel) — pulls parts that already exist elsewhere in the dataset onto the current view, instead of recreating them.
- **Insert Smart Stream / Smart Check View** — trace and pull in a chain of already-existing, already-connected parts onto a view, filling gaps between what's on the diagram and what's actually recorded in the dataset.

Practically, this means the safe habit is: capture information as Parts and Connectors (via any view, any command, any import), and treat every view as something you could delete and rebuild without losing anything. Part 3.10 of this manual walks through exactly that, concretely, as proof.

2.3 The Catalog of Parts Common to Every Discipline

The left-hand **Elements** panel is a single, shared palette of part types — not a separate toolset per discipline. The same palette a business analyst drags a Business Process from is the palette a data modeler drags a Data Entity from and an infrastructure architect drags a Technology Node from. Every type in it is grounded in the ArchiMate 3.x standard (plus a handful of DyCAD-specific additions like Data Entity Details and UI dashboard elements), grouped by both **Source** — TOGAF, ArchiMate, or Other — and by **discipline group** — Business, Application, Data, Technology, Strategy/Motivation, Implementation/Migration, and General/Other/UI.

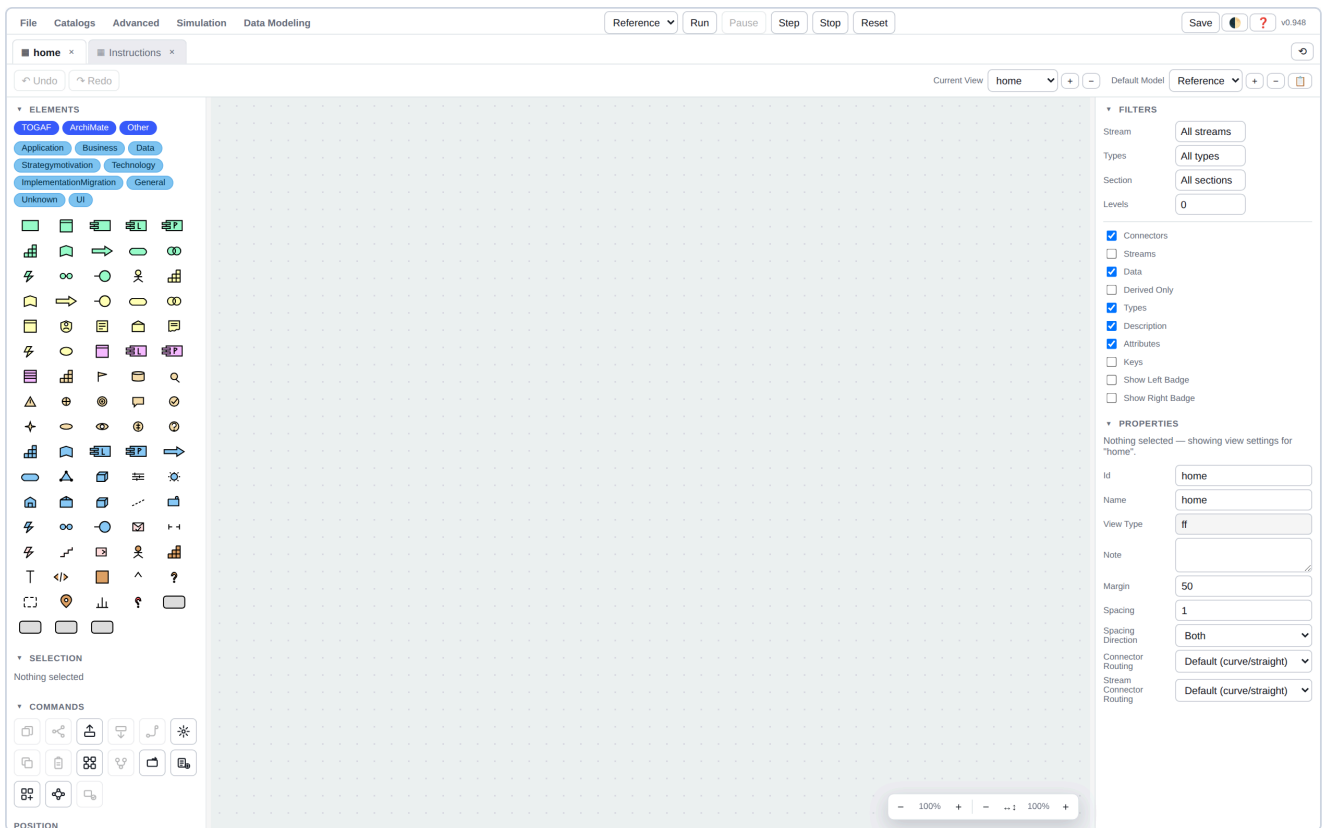


Fig. 2.1 — The Elements toolkit (left sidebar) on a blank canvas. Source chips (TOGAF / ArchiMate / Other) and discipline-group chips (Application / Business / Data / Technology / ...) filter the same shared palette every discipline draws from — this is not five separate toolboxes stitched together.

Because every discipline pulls from the same catalog into the same dataset, a connector can legitimately run directly from a Data Entity to the Business Capability that depends on it, or from an Application Component to the Technology Node it's deployed on, with no translation step and no separate tool to reconcile against. The "catalog" isn't a metaphor here — it is quite literally one JSON configuration (`public/custom.json`) that defines every element type, every valid relationship pair, and every Generate View/Generate Industry template the rest of the app uses, so a new part type or relationship becomes available to every discipline the moment it's added, not just one.

2.4 Streams: Tagging Cross-Cutting Flows

A **Stream** is a tag attached to a set of parts and connectors to say "these belong to this named business flow" — e.g. every part involved in delivering the "Order Management" capability, cutting straight across the Business/Application/Data/Technology layers described above. Streams are what the **Generate Stream** and **Remap** commands key off: a stream *template* describes the shape a flow should take (which element types, in what order, how they connect), and DyCAD builds or reorganizes a set of tagged parts to match it, rather than requiring every box and line to be drawn and connected by hand. This is covered hands-on in Part 3.7.

Guided Walkthrough

This walkthrough follows one continuous path through the application: load a small populated example, generate a full set of standard views from it, look at what came out, edit something, script a view from the console, and prove that the views themselves are disposable.

3.1 First Launch

Opening DyCAD (served locally, or hosted) lands you on two tabs: a blank **home** canvas underneath, and an **Instructions** tab open on top of it — the in-app reference this manual expands on. Nothing is saved automatically anywhere; **File > Save** downloads the current document as one JSON file, and **File > Load** brings one back. There is no server-side storage — the whole application is static files, and your model is a file on your own computer.

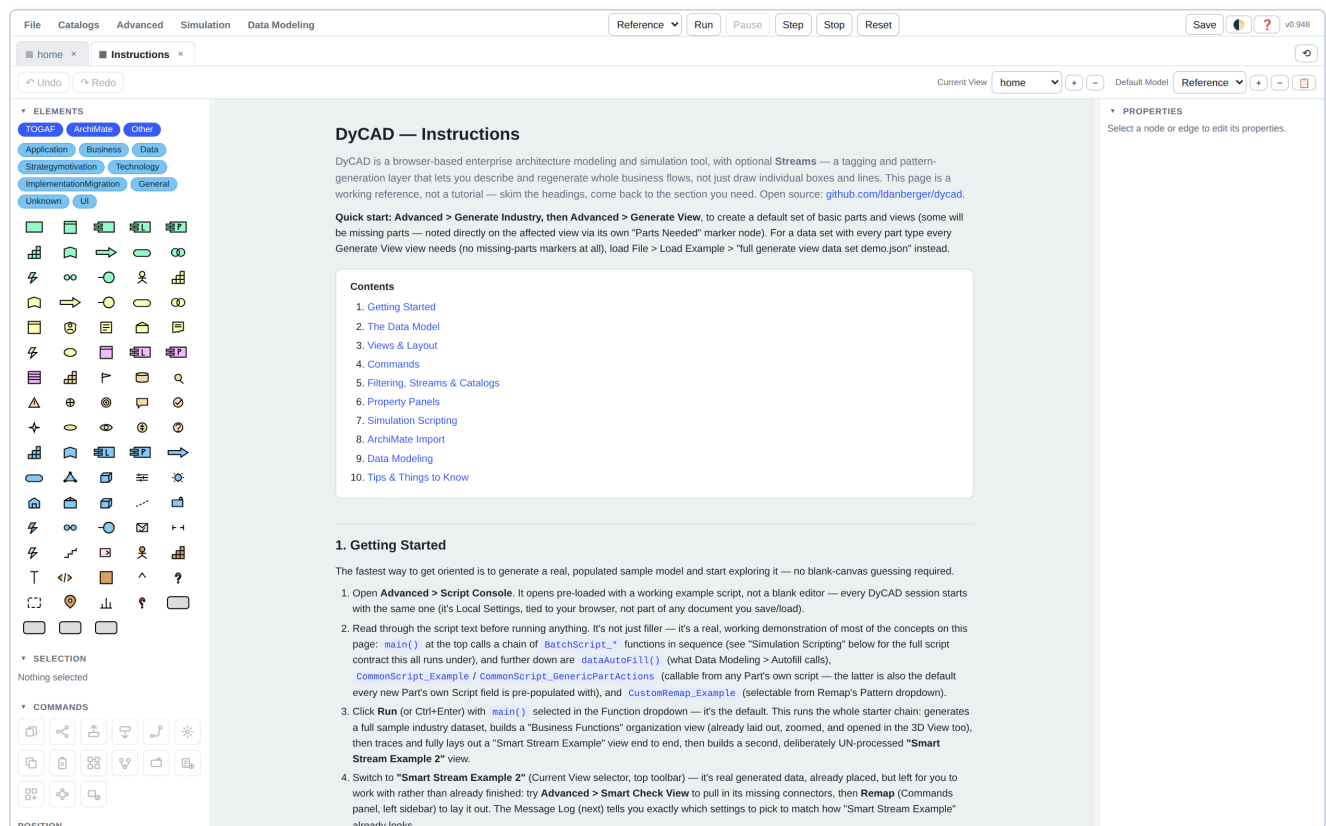


Fig. 3.1 — First launch: the Instructions tab, DyCAD's own in-app reference, opens automatically on top of a blank Home canvas.

3.2 Loading a Populated Example

Rather than start from an empty canvas, this walkthrough loads one of the bundled example datasets: **File > Load Example > "full generate view data set demo.json"** — a small reference dataset deliberately built to include at least one Part of every element type every Generate View starter view needs, so nothing comes up short.

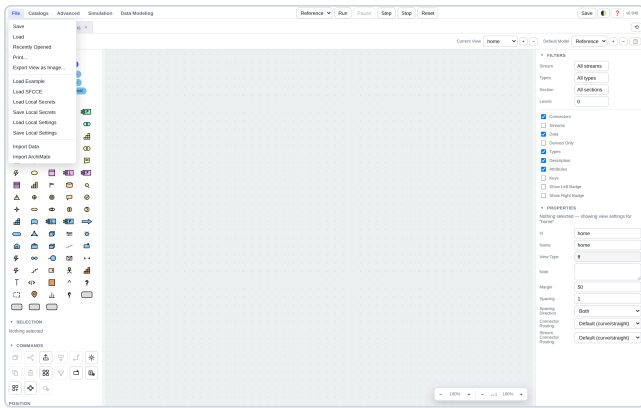


Fig. 3.2 — File menu.

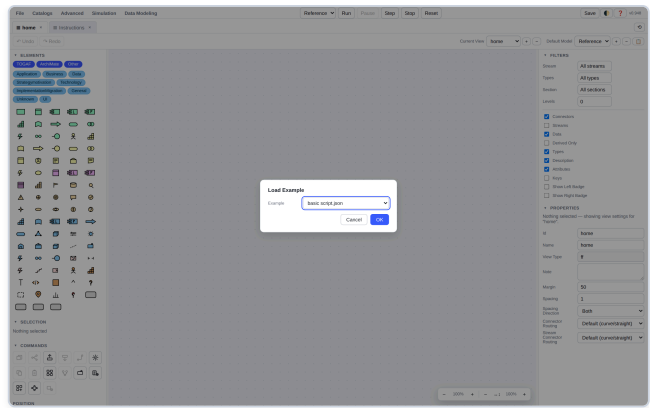


Fig. 3.3 — Load Example dialog, listing every bundled example.

Note: Loading a new document (or a new example) replaces the current one entirely. DyCAD asks first if there are unsaved changes worth keeping.

3.3 Browsing the Parts Catalog

With the example loaded, **Catalogs > Parts** opens a full, sortable table of every Part in the document — independent of any one diagram. This is the dataset described in Part 2, made directly visible: 43 parts, each with a Type, a Label, and the Model it belongs to, and not one of them yet placed on any view you've opened. The example ships with exactly one pre-built reference view (visible as a single row count elsewhere), but the parts themselves already fully exist, ready to be pulled onto whatever views you generate next.

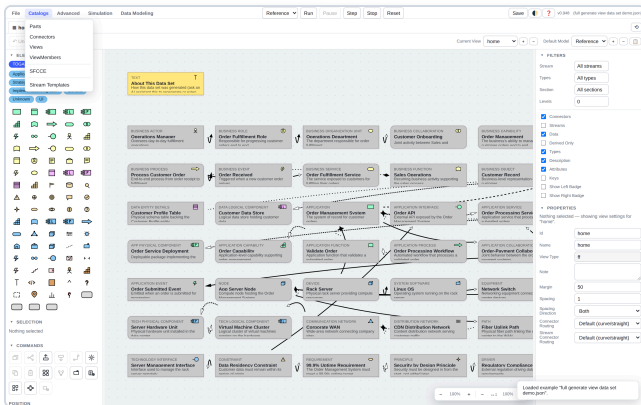


Fig. 3.4 — Catalogs menu: Parts, Connectors, Views, and ViewMembers each get their own table.

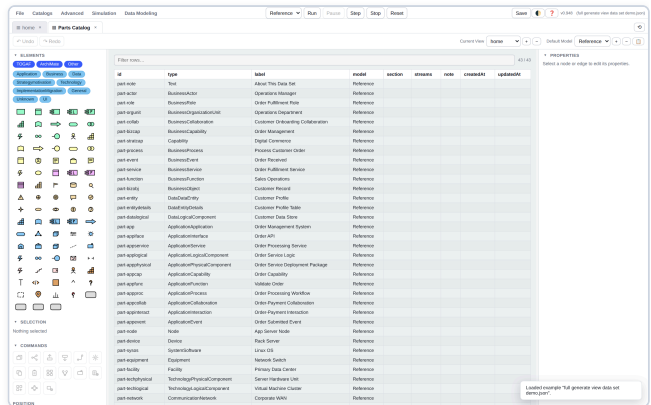


Fig. 3.5 — The Parts catalog: every Part in the document, one row each, spanning every discipline — actors, capabilities, processes, applications, data entities, technology components — in one table.

3.4 Generating Views From the Dataset

With 43 real Parts already in the dataset but only one view showing any of them, **Advanced > Generate View** is the command that turns "data that exists" into "diagrams you can actually look at." It builds one starter view per checked item, using only Parts and plain relationships already present in the chosen Model — it never invents new Parts. This is the regenerate-views-from-the-dataset mechanic from Part 2.2, in its most direct form.

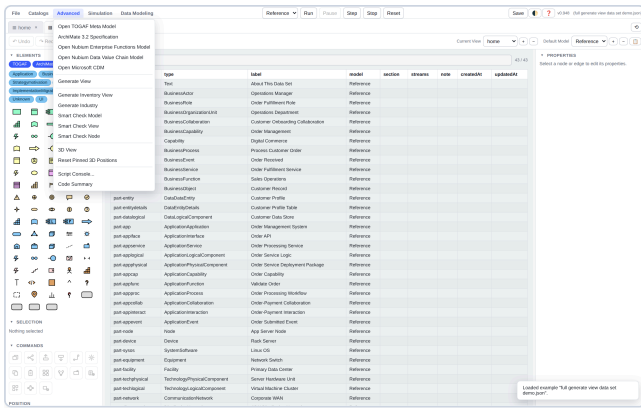


Fig. 3.6 — Advanced menu.

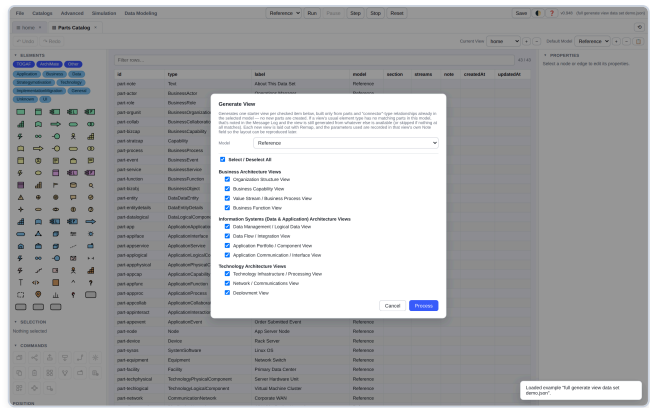


Fig. 3.7 — Generate View: 13 TOGAF-style starter views across four groups, all checked by default.

Clicking **Process** with every box checked (the default) builds all 13 views in one pass, each one already laid out via Remap, and records the exact parameters used in that view's own Note field so the layout can be reproduced later. If a view's usual element type has no matching Parts in the chosen Model, that gap is written to the Message Log by name and the view is still generated from whatever else is available.

3.5 Touring the Generated Views

A few of the 13 generated views, to show the same underlying dataset from different disciplinary angles — this is the "one dataset, many diagrams" idea made concrete.

Business Capability View

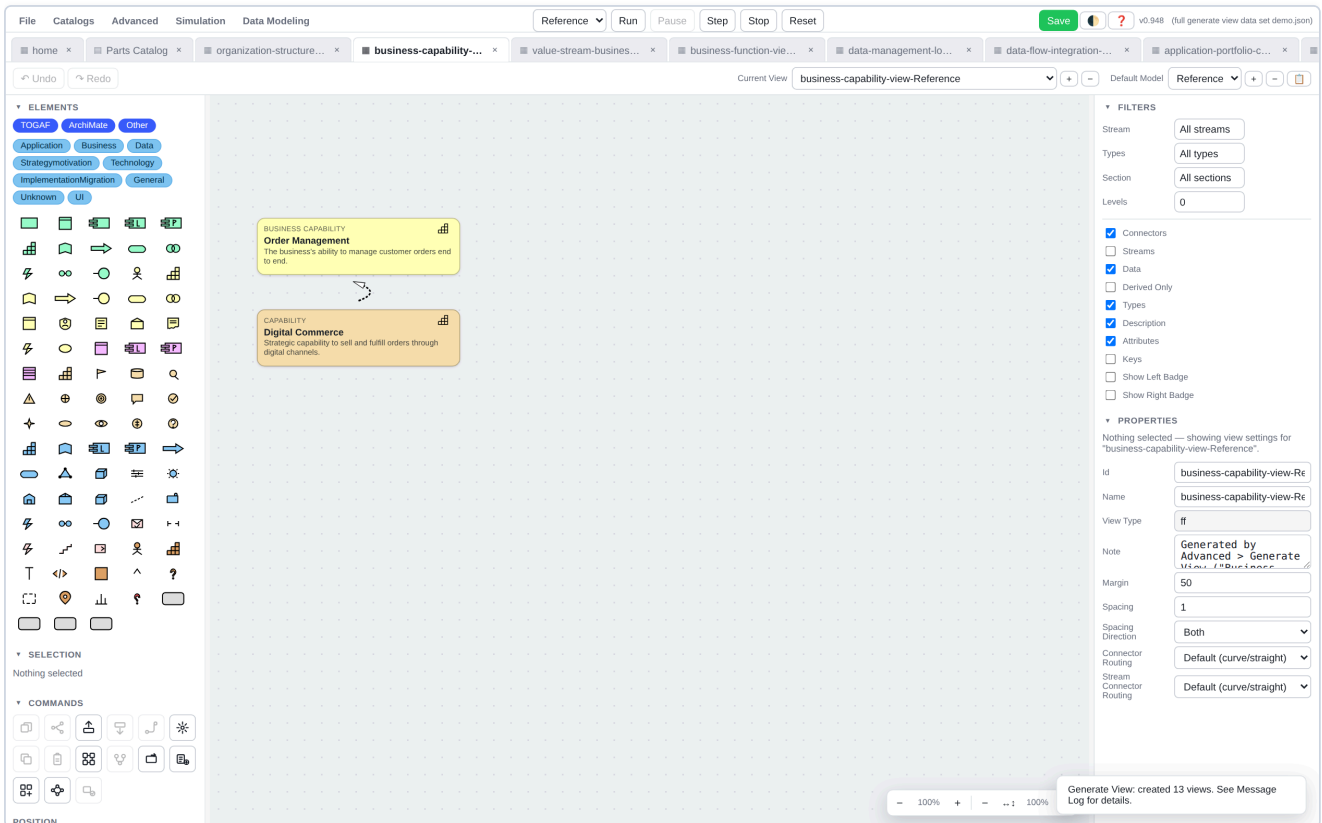


Fig. 3.8 — Business Capability View: "Order Management" (Business Capability) and "Digital Commerce" (Capability), pulled straight from the dataset with no manual placement.

Organization Structure View

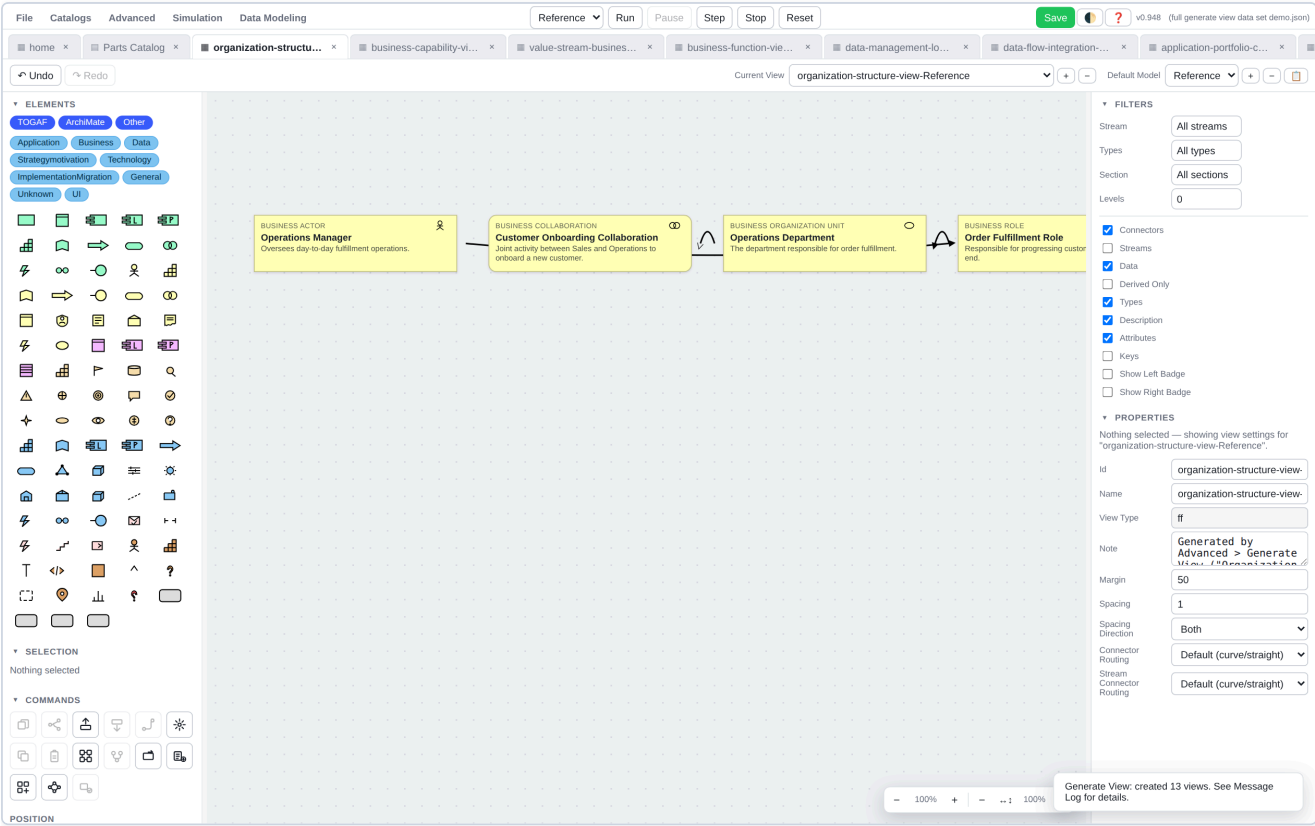


Fig. 3.9 — Organization Structure View: Business Actor, Business Collaboration, Business Organization Unit, and Business Role, already connected by their real relationships — the "Operations Manager" node here is selected in the next section.

Data Management / Logical Data View

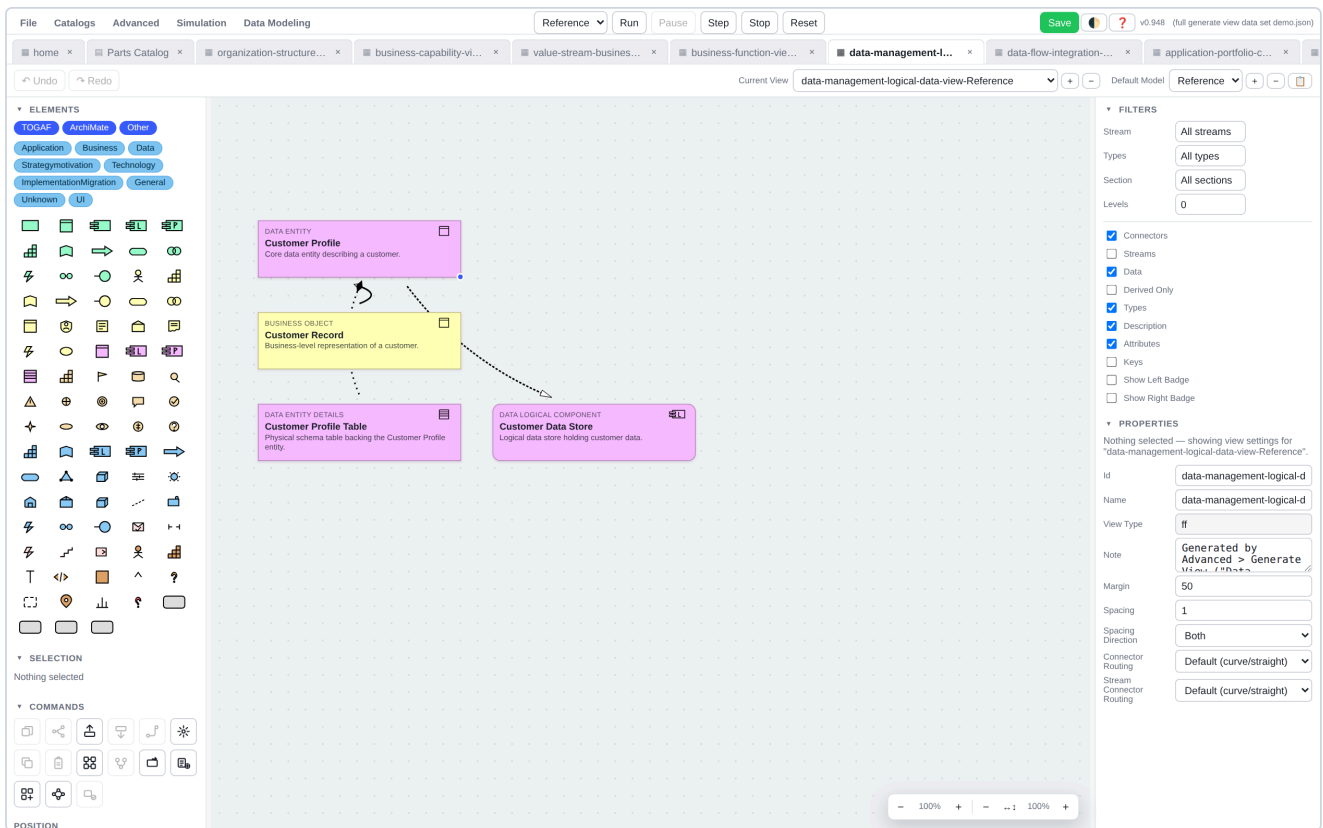


Fig. 3.10 — The Data layer of the exact same dataset — Data Entities and their logical components, generated by the identical command as the business-layer views above.

Application Portfolio / Component View

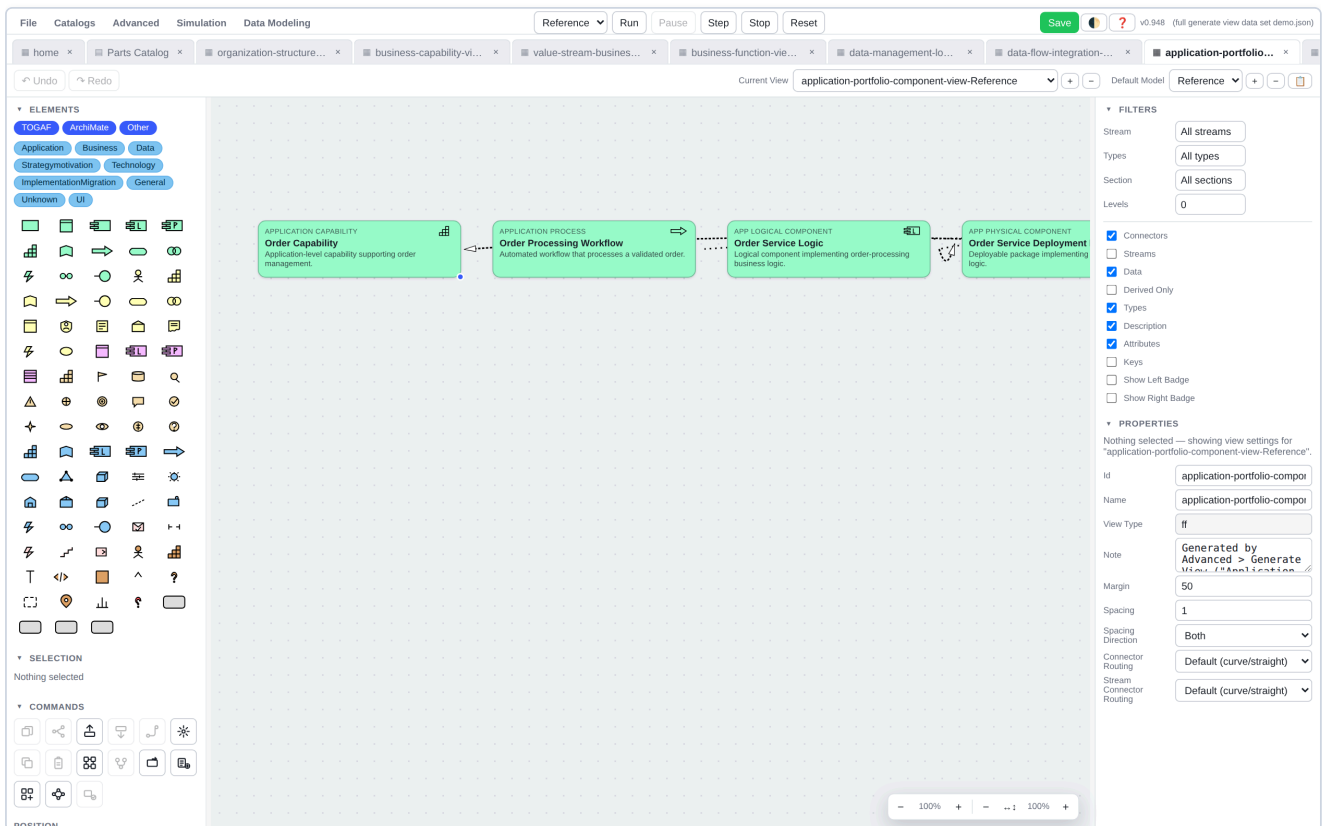


Fig. 3.11 — Application Capability through to its physical deployment package, one connected chain.

Technology Infrastructure / Processing View

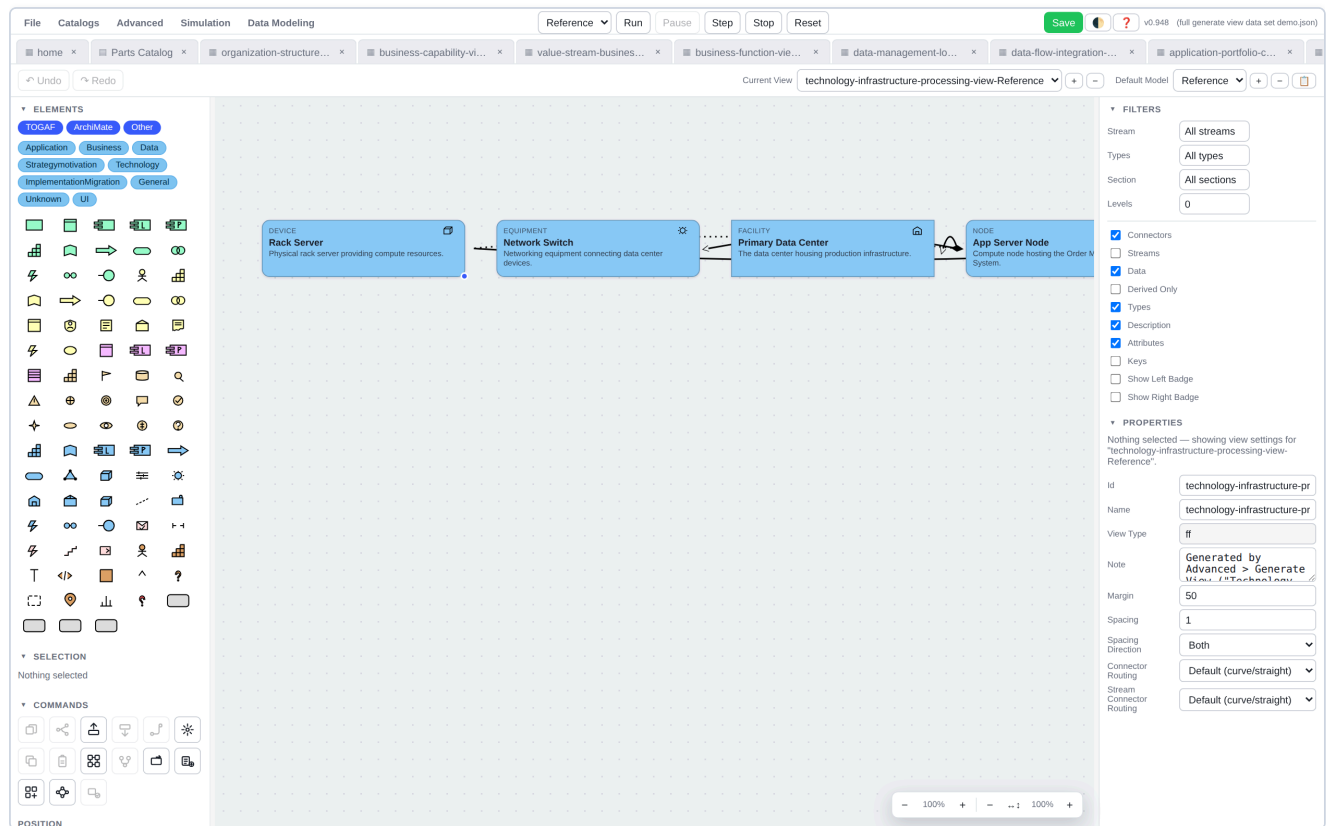


Fig. 3.12 — The Technology layer — Nodes, Devices, System Software — completing the chain from business capability down to physical infrastructure, all traceable through the shared dataset.

3.6 Editing a Part: the Property Panel Split

Clicking a node on any view opens the property panel on the right, and it is deliberately split into two sections that mirror the Part/ViewMember distinction from Part 2.1:

- A **Pinned / View** section at the top — the fields that belong to *this view's placement*: x/y position, Fill Color, Order, whether it's flagged External.
- A **Root Properties** section below — the fields that belong to the underlying Part itself, shared across every view that shows it: Id, Type, Label, Raw Label, Model, Streams, Script.

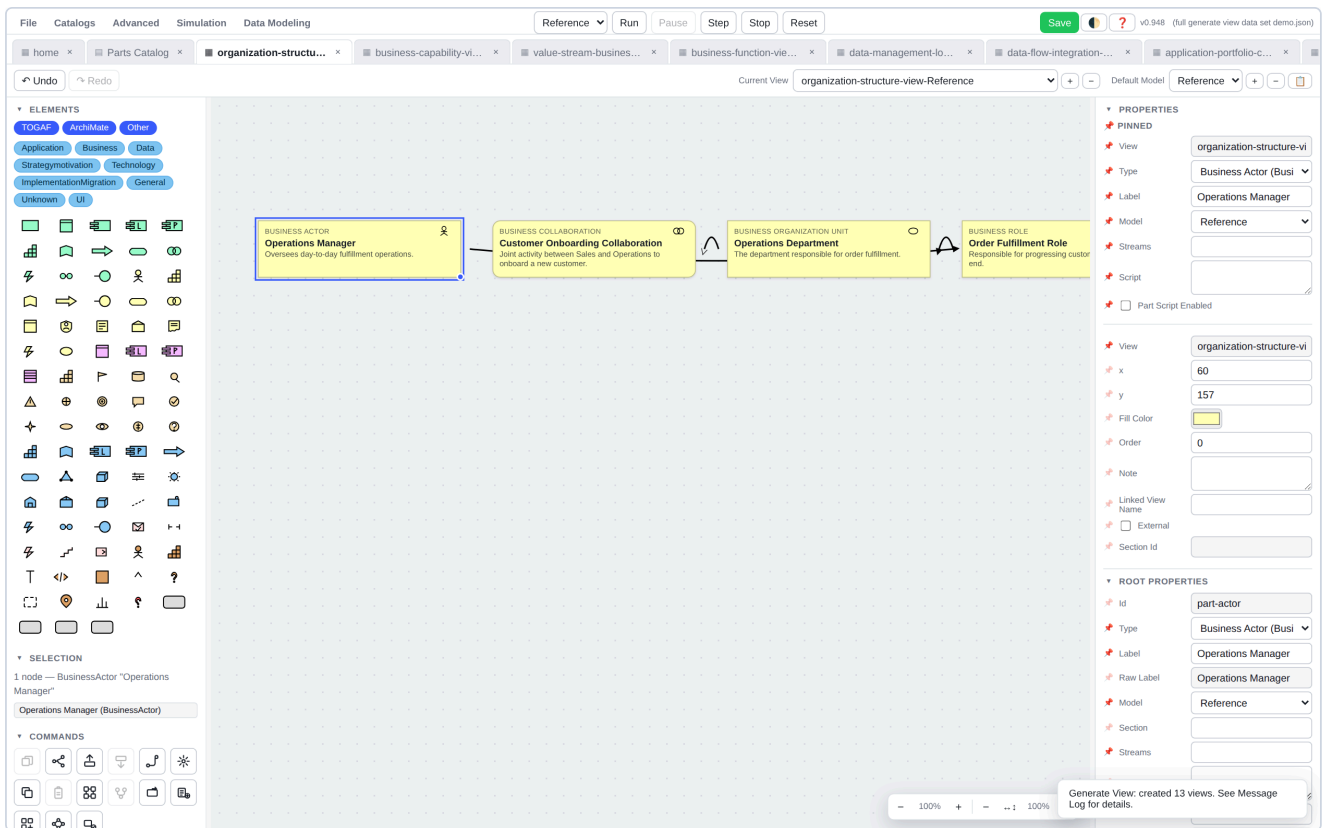


Fig. 3.13 — "Operations Manager" selected on the Organization Structure View. Top: this view's own placement (x, y, Fill Color, ...). Bottom, under **Root Properties**: the underlying Part's own Id, Type, Label, and Model — editing anything here would change it on every view that shows this same part.

Rule of thumb: if you want to change how something looks on *this one diagram*, edit the top section. If you want to correct the thing itself everywhere it appears, edit **Root Properties**.

3.7 Generate Stream and Stream Templates

The **Generate Stream** command (Commands panel, left sidebar) builds a whole tagged flow from a named template in one step: pick a stream template, name the capability/entity involved, and DyCAD creates and connects the matching set of parts — reusing anything that already matches by name and type instead of duplicating it.



Fig. 3.14 — The Commands panel (left sidebar): one icon button per command, enabled or disabled depending on the current canvas selection.

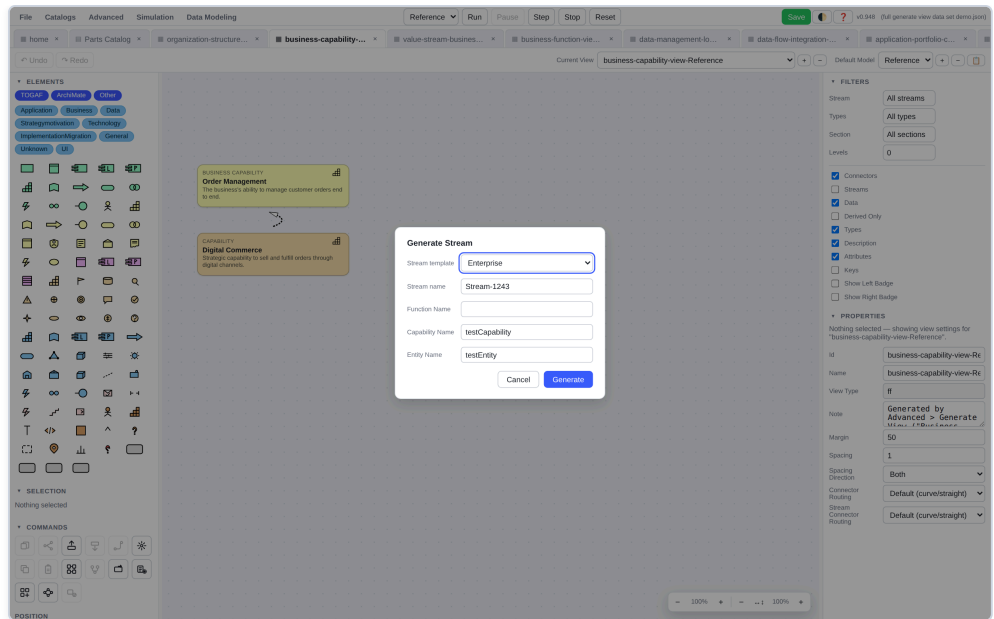


Fig. 3.15 — Generate Stream: pick a template (here, "Enterprise"), name the stream, and DyCAD builds the matching chain of parts and connectors.

Every named template is browsable read-only via **Catalogs > Stream Templates** — its element-type chain in order, and its "passive" from/to pairs — before you commit to using it.

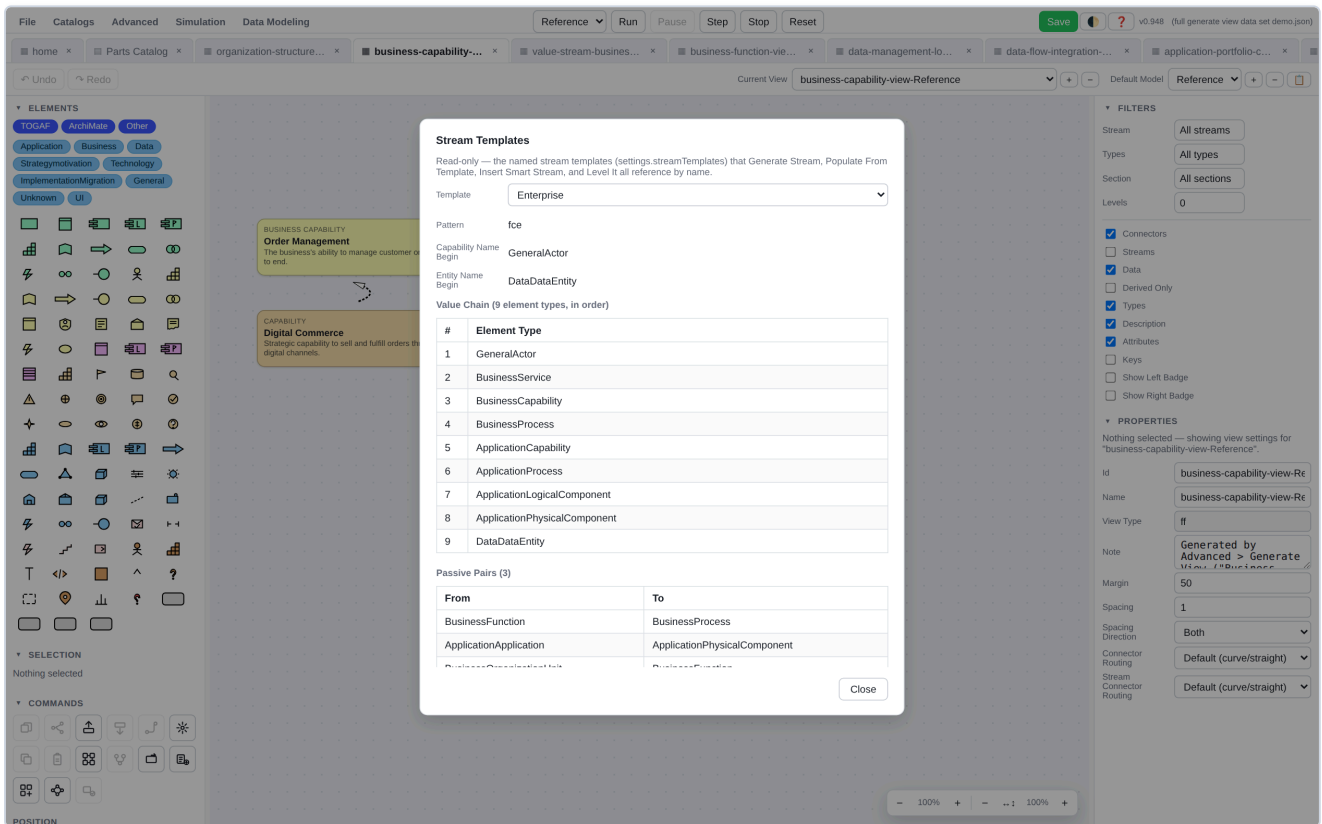


Fig. 3.16 — Stream Templates catalog: a read-only viewer for every named stream template Generate Stream, Populate From Template, Insert Smart Stream, and Level It can reference by name.

3.8 Console Scripting: Building and Remapping a View

Everything in this walkthrough so far was driven by menu items and dialogs. The exact same operations — generating data, creating a view, populating it, laying it out with Remap — are also callable directly from code, via **Advanced > Script Console**. Scripting in DyCAD is not limited to the per-tick simulation scripts described in Part 4.6: the console is a general-purpose batch-automation surface with full access to the same functions the UI's own dialogs call, so one script can chain several operations that would otherwise take several separate dialogs.

Every session opens the console pre-loaded with a real, working script, not a blank editor. Its `main()` function calls a chain of helper functions in sequence:

- `BatchScript_QuickStart()` — calls `generateIndustry(app, null, false)` to build a full sample dataset (parts and connectors only, not yet placed on any view), creates a new section-based view named "Business Functions" (`store.addView('Business Functions', 'org')`), fills it via `populateFromTemplate(app, tab, 'Enterprise Functions')` , and lays it out with `remap(app, tab, { pattern: 'default' })` — the same three operations Generate Industry, Populate From Template, and Remap perform interactively, chained into one function call.
- `BatchScript_InsertSmartStreamExample()` — creates a new freeform view named "Smart Stream Example" and calls `insertSmartStream` directly, tracing every part connected (by Stream-type connector, in both directions, out to any Data Entity) to the Business Function named "Production," narrowed to a specific list of element types to show.
- `BatchScript_SmartCheckViewExample()` and `BatchScript_RemapExample()` — repair and lay out that same "Smart Stream Example" view: `smartCheckView` with missing connectors and derive-hidden-connections both on, then `remap` with the 'layered' pattern plus Minimize Crossings and Minimize Connector Length.
- `BatchScript_InsertSmartStreamExample2()` — repeats the same trace with a broader element-type list into a second, deliberately *unprocessed* "Smart Stream Example 2" view, left for a person to finish interactively — `main()` 's own closing log message names the exact Smart Check View/Remap settings needed to bring it to the same finished state as the first.

Clicking **Run** (or Ctrl+Enter) with `main()` selected — the default — runs this entire chain in one click: one generated dataset, four views built and laid out, with only the last deliberately left as a hands-on exercise.

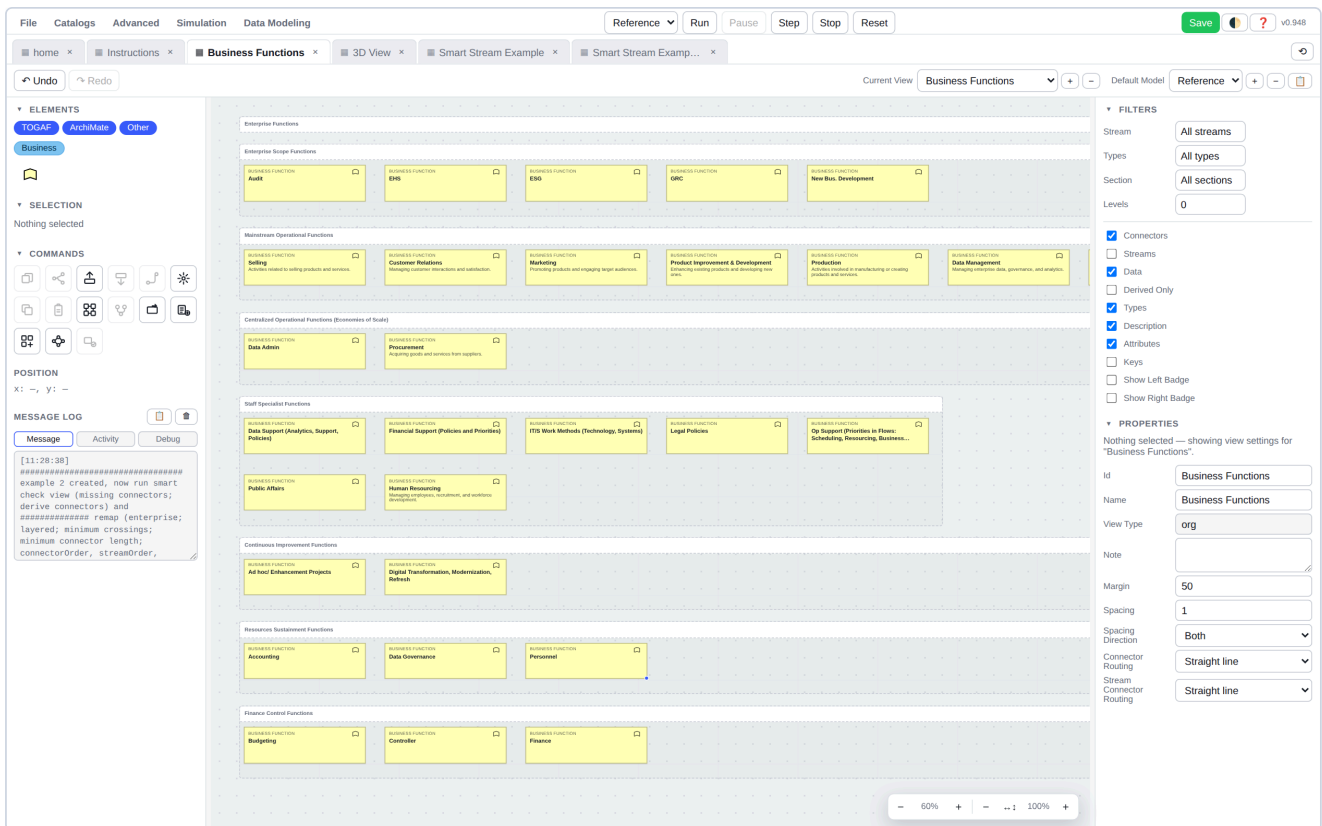


Fig. 3.17 — The "Business Functions" view `main()` builds: every generated Business Function grouped into its own Section as a fixed grid, Connector Routing set to Straight line. An example of a structured view useful in a joint session with a business client, to drag/drop/remove/add business functions and organizational units quickly — each section only accepts the element types it's meant to, and dragging a node snaps it straight into the nearest open cell.

3.9 The 3D View

Advanced > 3D View renders every Part and Connector in the document (or, scoped via **View Scope**, just one 2D view's own placements) as a rotatable, zoomable 3D scene. Parts stack into layers by element type and band by Section/Stream, so the same dependency chain that runs across the flat 2D views in 3.5 is visible here as one continuous structure spanning every discipline at once.

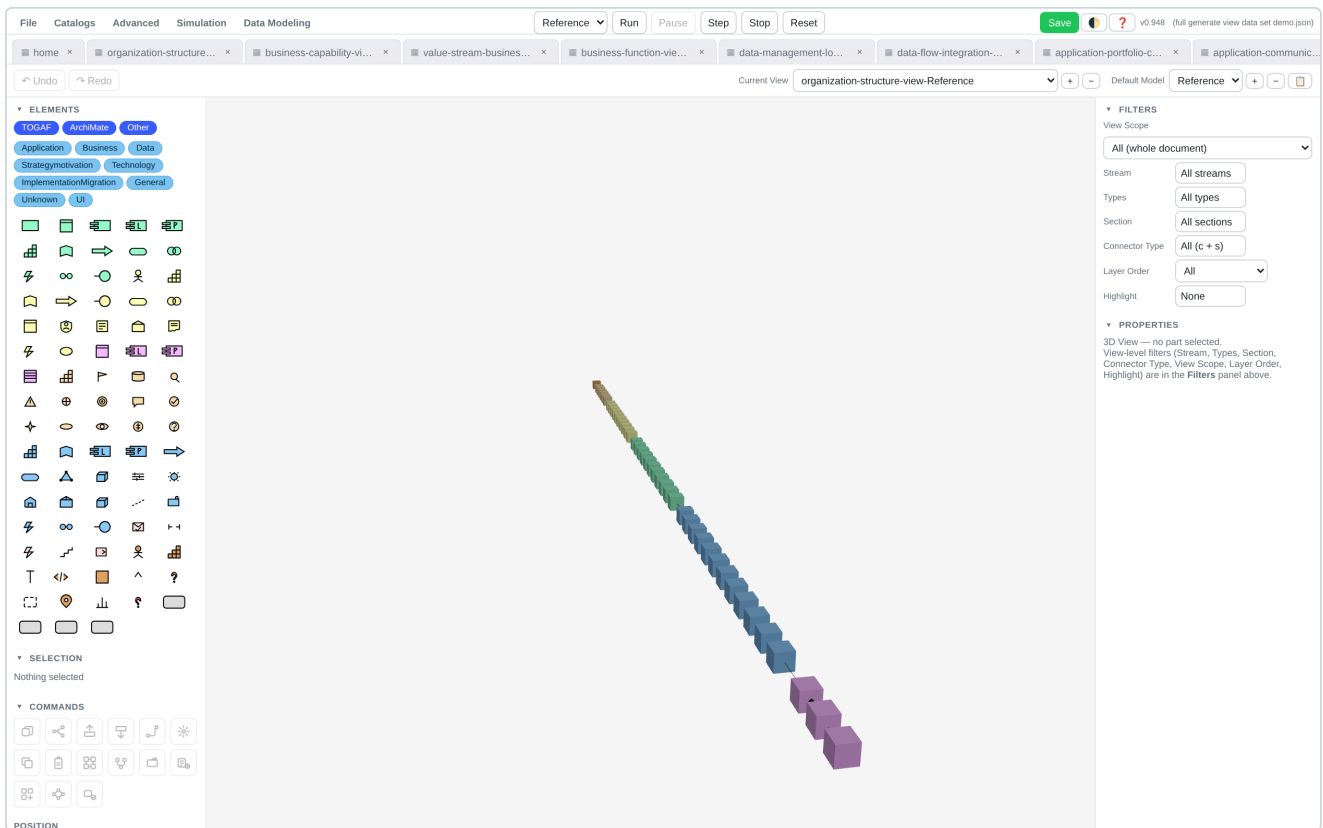


Fig. 3.18 — The 3D View, zoomed and rotated to show the full document. With this particular reference dataset (one or two parts per element type, so every Generate View starter view has exactly what it needs and no more), each type's own layer band is thin — a larger real-world dataset produces visibly wider bands per layer rather than this narrow diagonal chain.

3.10 Proof: Delete a View, Regenerate It

To make Part 2.2's claim concrete rather than theoretical: closing the **business-capability-view-Reference** tab (the × on its tab, or Delete on the View catalog row) removes only that view and its ViewMembers. The "Order Management" and "Digital Commerce" Parts shown in Figure 3.8 are untouched — they are still two rows in the Parts catalog, exactly as in Figure 3.5. Reopening **Advanced > Generate View**, checking just **Business Capability View**, and clicking **Process** again rebuilds an equivalent view from those same underlying Parts — same content, freshly laid out — because the view was never where the data lived in the first place. The same is true of every other view in this walkthrough: none of them are the "real" copy of anything; the Parts and Connectors catalogs are.

Reference

This part is a working reference, not a tutorial — skim the headings and come back to the section you need. It mirrors the depth of DyCAD's own in-app Instructions tab ([Advanced](#) menu's [?](#) icon), reorganized here for a printed manual.

4.1 The Data Model

See Part 2.1 for the conceptual explanation. In short: **Part** and **Connector** are the shared, model-level truth; **View** and **ViewMember** are per-diagram placement; **Model** is a namespace Parts/Connectors belong to (as-is vs. to-be, one business unit vs. another, simulated independently).

The **Default Model** selector (top toolbar) controls which model new parts/connectors are created into. Its **+/-** buttons add or remove a model entry (removing one keeps its parts/connectors — they just lose their model tag's registration); the copy-model button clones every part and connector tagged with the current model — including each part's own script — into a new model, ready for a fresh set of views to be built around either or both.

4.2 Views & Layout

View types

- **Free form** — an open canvas, place things anywhere. Most modeling happens here.
- **Section-based views** — the canvas is divided into fixed, named sections (a grid of cells), each restricted to certain element types; dragging a node snaps it into the nearest valid cell.

Remap: six layout patterns

Pattern	What it does
Default	Groups nodes by stream into rows, following the stream template's own ordering.
None	A plain sorted grid, no stream grouping.
Layered	Rows come from the directed connector graph instead of stream/group membership — a node's row is the fewest real-connector hops from a root.
Force-directed	Connected nodes pull into adjacent grid cells; unconnected clusters pack into their own regions. Best for "just show me how this is actually connected."
Centralize in Clusters	Like Force-directed, but repeats the "one central hub" idea per local cluster instead of once for the whole diagram — especially useful for one large, densely connected ERD.
Custom (Script Console)	Hands placement to a <code>CustomRemap_<Name>(ctx)</code> function you write in the Script Console.

Two optional cleanup passes, **Minimize connector crossings** and **Minimize connector length** (in that order), apply to the Default/None/Layered patterns. **Edge Assignment** pins a chosen element type to a numbered slot along the Top/Bottom/Left/Right edge of the layout instead of its normal row/column. **Align by section** (on by default, except on Force-directed/Custom) nudges two connected parts that share a Section into the same column, one row apart.

Connector routing

Style	Behavior
Default	A gentle curve or straight line, no obstacle avoidance.
Direct	Routes around any node in the way with straight segments.
Manhattan	Same, but every segment is horizontal or vertical only.

Routing only looks at obstacles near each connector's own path (not the whole diagram) — this keeps large views fast, at the cost of not rerouting around something on the far side of a busy canvas.

4.3 Commands

Available from the Commands panel (left sidebar) on a canvas view, and via right-click. Availability depends on the current selection.

Command	What it does
Generate Stream	Builds a whole flow from a stream template.
Duplicate Stream	Clones an existing tagged stream under a new name.
Split Node	Creates a sibling copy of a part and rewires its outgoing connectors.
Level Up	Opens a new parent view containing this view's nodes (or, for a single Data Entity Details node, creates/opens its Data Entity parent).
Level Down	Pushes selected nodes into a new sub-view, replacing them here with a single linked node.
Level It	Freeform views only — replaces a node connected across a template gap with the correct in-between-type node that already exists.
Merge	Combines two or more selected nodes into one.
Copy / Paste	Standard copy-paste, within or across views.
Remap	Reorganizes a view's layout — see 4.2.
Redraw	Recalculates the best node size for a view's actual content.
Add Existing	Brings parts that already exist elsewhere into this view.
Populate From Template	Fills a view from a page template matching its view type.
Insert Smart Stream	Freeform views only — traces a chain of existing parts into this view by element type, direction, and level depth.
Smart Check View / Node (Advanced menu)	Repairs gaps between a view and the underlying model: missing connectors, missing connected nodes, derived (implied) connections across hidden types.
Generate View (Advanced menu)	Builds the standard TOGAF starter view set from a chosen model — see Part 3.4 and Appendix A.
Generate Industry / Generate Inventory View (Advanced menu)	Bulk-generates a starter set of parts/connectors from the loaded industry dataset, or a single inventory view of the whole model.

4.4 Filtering, Streams & Catalogs

- **Stream filter** — show only parts/connectors tagged with a chosen stream.
- **Type filter** — show only chosen element types.
- **Section filter** — show only parts tagged with a chosen Section.
- **Connector levels** — how many extra hops of connected nodes to also show beyond direct matches.
- **Toolkit filters** (left sidebar) — Source chips (TOGAF/ArchiMate/Other) and discipline-group chips narrow the shared Elements palette itself.
- **Catalog tables** — Parts, Connectors, Views, ViewMembers: full sortable tables, click a header to sort, shift+click to add a secondary sort column.

4.5 Property Panels

Selecting a node or connector shows two layers, described hands-on in Part 3.6: the view-placement fields at the top, and the shared **Root Properties** below. The same split applies to the Parts/Connectors catalog tables, which show only the Root Properties side, since there's no one view's placement to speak of outside a diagram.

Changing a connector's Relationship or Streams also checks whether a related part-to-part "inventory" connector exists elsewhere in the model (from a Level Down or Smart Check View operation) and offers to keep both in sync.

4.6 Scripting: Simulation Ticks and the Script Console

DyCAD has two related but distinct uses for a script — worth keeping apart, since only one of them has anything to do with simulation:

- A **per-Part simulation script** runs once per simulation tick, modeling how a value or state changes over time on a live, ticking architecture.
- The **Script Console** (Advanced menu) is a general-purpose batch-automation surface, not limited to simulation at all — it has full access to the same functions the UI's own dialogs call (`generateIndustry` , `populateFromTemplate` , `remap` , `smartCheckView` , `insertSmartStream` , `findParts` , and direct `app / store` access), so a script can build a view, populate it from a template, and lay it out with Remap — all from code instead of clicking through several dialogs in sequence. Part 3.8 walks through a real, working example.

Per-Part simulation scripts

Any Part can carry a script that runs once per simulation "tick," letting you model how values or state flow and change across a live architecture rather than a static diagram. Turn it on per-part with **Part Script Enabled** in Root Properties (or bulk-toggle many at once via **Simulation > Manage Part Scripts...**).

Each tick, a script receives a `ctx` object exposing `ctx.part` (the part record), `ctx.inputs` (one entry per incoming connector, including the neighbor's last-known-good value and whether it just changed), and `ctx.outputs` (the part's own outgoing connectors, structural only — one return value broadcasts identically to all of them). A script's return value becomes this part's value for the tick; parts with a live value show a small colored status marker on canvas (green normal, blue pulsing on change, red on error) and in the 3D View.

The Script Console

The Script Console is where per-Part scripts, batch functions (like the Part 3.8 example), and Custom Remap functions are all written and run — it opens pre-loaded with a working example on every session, ready to run with one click (**Run**, or Ctrl+Enter) rather than a blank editor.

4.7 ArchiMate Import

File menu > Import ArchiMate reads a standard ArchiMate 3.0 Exchange Format file and brings in its elements, relationships, and views as DyCAD Parts/Connectors/Views. Two ArchiMate conventions get special handling: **Junctions** (AND/OR routing points) are dissolved — every real source connects directly to every real target it fed through the junction, noting which logic it represented; and **nested shapes** (a Composition/Aggregation drawn as one shape inside another rather than a line) are detected and turned into a real connector, noted as inferred from nesting. Re-importing the same file updates existing records in place by their own stable ids rather than duplicating them.

4.8 Data Modeling (Entity-Relationship)

The **Data Modeling** menu adds ERD detail — typed attributes, primary/foreign keys, crow's-foot notation — on top of the ordinary Data Entity element, without changing what a Data Entity means anywhere else in the model.

- **Add/Edit Entity Details** — select a Data Entity and this creates (or reopens) a linked **Data Entity Details** node with its own Attributes table (Name, Data Type, Null, PK). The reverse works too: Level Up on a Data Entity Details node creates/reopens its parent Data Entity.
- **Autofill** — scaffolds a whole view's tables and relationships in one click: a numeric PK attribute plus Name/Description for every untouched table, and From/To attribute wiring for every unconfigured Data connector.
- **Crow's-foot relationships** — drag between two Data Entity Details nodes to create a typed **Data (d)** connector automatically; a matching foreign key attribute is created on the target table if the source has a primary key.
- **Import/Export DDL** — reads a `.sql` file's `CREATE TABLE` statements (a common subset) into Data Entity Details nodes and Data connectors on a new view; Export generates DDL text back from the current view.
- **Auto-Detect Connectors** — scans the whole model for likely but not-yet-connected relationships (by pasted DDL foreign keys, or by field-name convention like `CustomerId` matching Customer's primary key) and shows a checkable preview before creating anything.

4.9 Tips & Things to Know

- **Undo/Redo** (Ctrl+Z / Ctrl+Shift+Z) covers the full model, not just the current view — safe even though views share underlying parts and connectors.
- **The Log area** (left sidebar) has three tabs — Message, Activity, Debug — each an independent record. The Message tab records every warning/rejection the app shows, not just the ones caught as a popup; check it after any bulk operation.
- **Rejections give a specific reason** — e.g. dragging a node into a section that doesn't allow its type names the section and what it does allow, not a generic failure message.
- Force-directed layout can't guarantee every edge in a cycle lands in an adjacent grid cell — a hard 2D layout limit, not a bug. Obstacle-avoiding routing only checks nearby obstacles, for performance on large diagrams — also intentional.

4.10 Keyboard Shortcuts

Shortcut	What it does
Ctrl/Cmd + Z	Undo (per-tab history).

Shortcut	What it does
Ctrl/Cmd + Shift + Z or Ctrl/Cmd + Y	Redo.
Ctrl/Cmd + C	Copy the current canvas selection.
Ctrl/Cmd + V	Paste.
Delete / Backspace	Delete the current selection — removes just this view's placement(s); if nothing else references the part/connector, you're separately asked whether to delete it from the model entirely.
Ctrl/Cmd + Enter	Run the Script Console's selected function.

Appendix

A. Generate View: the Full TOGAF Starter Set

The complete list of views **Advanced > Generate View** can build in one pass, grouped exactly as they appear in the dialog (Fig. 3.7). Each is built only from Parts/plain-relationships already in the chosen Model.

Business Architecture Views

View	Needs at least one...
Organization Structure View	Business Actor or Business Role
Business Capability View	Business Capability
Value Stream / Business Process View	Business Process
Business Function View	Business Function

Information Systems (Data & Application) Architecture Views

View	Needs at least one...
Data Management / Logical Data View	Data Entity
Data Flow / Integration View	Application and Application Interface
Application Portfolio / Component View	Application
Application Communication / Interface View	Application and Application Interface

Technology Architecture Views

View	Needs at least one...
Technology Infrastructure / Processing View	Node or Device
Network / Communications View	Communication Network
Deployment View	Node or Device

Cross-Cutting & Governance Views

View	Needs at least one...
Security & Control View	Constraint or Requirement
Standards & Interoperability View	Principle or Requirement

B. Where to Learn More

- The in-app **Instructions** tab (opens automatically on launch, or via the  icon) — the living source this manual was built from; always matches the version you're running.
- `DESIGN_DOCUMENT.md`, in the project repository — the technical/architectural reference behind every mechanism described here.
- `js/version.js`'s changelog comments — the authoritative per-release history of why the application behaves the way it does.
- github.com/ldanberger/dycad — source, issues, and the latest build.